

Deferrable Server

Understanding and Study Guide

Not a seminar paper - a learning document for Milestone 1 preparation

Name	MD Rafiul Islam
Student ID	1232714
Course area	Real-Time Systems
Topic	Deferrable Server
Document type	Understanding guide and preparation notes

Purpose of this document

This guide explains the Deferrable Server topic in a student-friendly but technical way. It is written to help understand the algorithm, prepare Milestone 1, and later write an independent seminar text without copying AI output or source text. It includes concepts, formulas, examples, comparison, limitations, source-use guidance, and a checklist.

Important: this is not the final seminar paper. It is a study document for understanding and planning.

Contents

- 1. What problem Deferrable Server solves
- 2. The basic intuition
- 3. Real-time task model and notation
- 4. Deferrable Server parameters
- 5. How the algorithm works step by step
- 6. Timeline example
- 7. Comparison with Polling Server and Sporadic Server
- 8. Schedulability, interference, and limitations
- 9. Runtime, memory, and implementation overhead
- 10. Application example for embedded systems
- 11. How to use sources legally and scientifically
- 12. Milestone 1 writing guidelines
- 13. Open questions and checklist
- References and source-use notes

1. What problem Deferrable Server solves

A real-time system is not only about producing a correct value. It must also produce the value at the correct time. In such systems, a late result can be treated as a failure, especially when control, safety, communication, or embedded timing is involved.

Many systems contain two kinds of work:

- **Periodic tasks:** tasks that arrive regularly, for example a sensor read every 10 ms or a motor-control update every 20 ms.
- **Aperiodic tasks:** tasks that arrive irregularly, for example a button press, diagnostic command, network message, alarm, or user request.

The scheduling problem is a trade-off. Periodic tasks often have hard deadlines and must be protected. Aperiodic tasks do not arrive at fixed times, but users or external devices may still expect fast response. If aperiodic jobs only run in the background, their response time may be poor. If they are given too much priority, periodic tasks may miss deadlines.

Core problem: How can the system respond quickly to irregular aperiodic requests while still keeping periodic real-time tasks schedulable?

2. The basic intuition

The Deferrable Server is a server-based scheduling technique. A server is like a controlled helper task that reserves a limited amount of processor time for aperiodic jobs. It does not give unlimited CPU time to aperiodic work. Instead, it gives a fixed execution budget during a fixed server period.

A simple analogy is a teacher who reserves 10 minutes every hour for student questions. If no student asks a question exactly at the beginning of the hour, the 10 minutes are not immediately lost. The teacher can still use the remaining reserved time later in that hour. This is similar to Deferrable Server: unused capacity can be held and used later within the current server period.

Key idea: Deferrable Server preserves unused server budget within the current period, so an aperiodic request that arrives later can still be served quickly.

This is different from a simple Polling Server, where the server may waste its capacity if no aperiodic request is available when it is polled. Deferrable Server is therefore better for response time, but it can create extra timing interference for periodic tasks.

3. Real-time task model and notation

A periodic real-time task is often written as:

```
tau_i = (C_i, T_i, D_i)
```

- **C_i**: worst-case execution time of task i.
- **T_i**: period of task i, meaning how often the task is released.
- **D_i**: relative deadline of task i.

A simple utilization estimate for periodic tasks is:

```
U_periodic = sum(C_i / T_i)
```

Utilization gives a quick view of CPU load. However, utilization alone is not always enough to prove schedulability, especially when Deferrable Server behavior causes special interference. For deeper analysis, response-time analysis or time-demand analysis may be needed.

4. Deferrable Server parameters

The Deferrable Server itself can be represented by two main parameters:

```
DS = (C_s, T_s)
```

- **C_s**: server capacity or server budget. This is the maximum processor time reserved for aperiodic jobs in one server period.
- **T_s**: server period. This defines how often the server budget is replenished.
- **U_s = C_s / T_s**: utilization reserved by the server.

Example: if C_s = 5 ms and T_s = 20 ms, the server reserves up to 5 ms of CPU time every 20 ms.

```
U_s = C_s / T_s = 5 ms / 20 ms = 0.25 = 25%
```

This does not mean the server always uses 25% of the CPU. It means the server is allowed to use up to that amount if aperiodic work arrives.

5. How the algorithm works step by step

The Deferrable Server can be understood as a budget-control mechanism. The server has a current budget value. Aperiodic jobs can be executed only while this budget is greater than zero.

1. At the start of each server period, the server budget is replenished to C_s.
2. If one or more aperiodic jobs are waiting and the server has budget, the server may execute them according to its priority.
3. While the server executes an aperiodic job, its budget decreases.
4. If the budget reaches zero, the server cannot execute more aperiodic jobs until the next replenishment.
5. If no aperiodic job is ready, the unused budget is not immediately lost. It is preserved until later in the same server period.
6. At the next server period, the budget is restored again according to the replenishment rule.

Simplified pseudocode:

```
At each server period boundary: budget = C_s
When an aperiodic job arrives: if budget > 0 and
server has scheduling priority: run the aperiodic job
budget = budget - execution_time_used
else: keep the job waiting
If no aperiodic job is ready: keep remaining budget until the end of the
```

current period when budget = 0: wait until next replenishment

6. Timeline example

Consider a server with $C_s = 5$ ms and $T_s = 20$ ms. The server gets 5 ms of budget in every 20 ms period. Suppose there is no aperiodic job at time 0, but one arrives at time 8 ms.

Time: 0 ms 8 ms 13 ms 20 ms |-----|-----|-----| Budget: 5 ms kept job arrives budget used replenish

Polling Server idea: If no job is ready at the polling time, budget may be wasted.

Deferrable Server idea: Budget can be kept, so the job arriving at 8 ms can still be served.

This example shows why Deferrable Server can give better response time for aperiodic jobs. The aperiodic job does not need to wait until the next server period if budget is still available in the current period.

7. Comparison with Polling Server and Sporadic Server

Method	Budget behavior	Advantage	Main concern
Polling Server	Budget is available at periodic polling points. If no aperiodic job is ready, capacity can be wasted.	Simple and easier to analyze.	Poor response time if requests arrive just after polling or after budget is lost.
Deferrable Server	Budget is replenished periodically and unused budget can be preserved within the current period.	Better aperiodic response time than simple polling.	Preserved budget can create extra interference with periodic tasks.
Sporadic Server	Consumed budget is replenished later based on consumption rules, not simply reset like DS.	Controls interference more carefully while still serving aperiodic jobs.	More complex implementation and bookkeeping.

Simple summary: Polling Server may waste budget, Deferrable Server preserves budget within the current period, and Sporadic Server uses a more controlled replenishment mechanism. Deferrable Server is powerful but must be checked carefully because its preserved budget can influence periodic tasks.

8. Schedulability, interference, and limitations

Schedulability means that all real-time tasks can meet their deadlines under the chosen scheduling algorithm and task assumptions. Deferrable Server is attractive because it improves aperiodic response time, but it can also make schedulability analysis more difficult than a simple periodic task model.

The main reason is budget preservation. Because the server can keep unused budget and execute later in the same period, it may interfere with lower-priority periodic tasks at a less convenient time. In a worst-case scenario, the server may execute near the end of one period and again after replenishment at the beginning of the next period. This can create a burst of server execution that affects periodic tasks.

- If C_s is too large, the server may improve aperiodic response time but increase the risk for periodic tasks.
- If C_s is too small, periodic tasks are safer, but aperiodic tasks may still wait too long.
- If T_s is too long, aperiodic jobs may wait for the next useful budget.
- If T_s is too short, the system may experience more frequent scheduling and timer overhead.
- Deferrable Server cannot be treated blindly as just another normal periodic task in all analyses.

Exam-style answer: Deferrable Server improves responsiveness by preserving unused capacity, but the same preserved capacity can increase interference. Therefore, server parameters must be chosen carefully and schedulability must be verified.

9. Runtime, memory, and implementation overhead

Overhead type	What causes it	Why it matters
Runtime overhead	Checking ready queues, updating budget, handling timer events, context switches.	Too much overhead can reduce the effective CPU time available for real tasks.
Memory overhead	Storing server budget, replenishment time, aperiodic job queue, and task metadata.	Usually small, but still important in embedded systems.
Implementation complexity	Managing budget preservation and preventing uncontrolled execution.	Incorrect implementation can break real-time guarantees.

Compared with Sporadic Server, Deferrable Server is easier to understand and implement, but it can be harder to analyze safely because of its preserved-capacity behavior. Compared with Polling Server, it has better response-time behavior but more possible interference.

10. Application example for embedded systems

A good application example is an embedded controller that has both periodic control tasks and irregular external commands.

Task	Type	Example timing	Purpose
Sensor sampling	Periodic	Every 10 ms	Read sensor values.
Motor control	Periodic	Every 20 ms	Update actuator command.
Status logging	Periodic/soft	Every 50 ms	Store diagnostic data.
Network command	Aperiodic	Unknown arrival time	Process external request.
Emergency message	Aperiodic	Unknown arrival time	React quickly to an event.

Without a server, the aperiodic commands might wait until the CPU is free. With a Deferrable Server, a controlled part of the CPU is reserved for such commands. The goal is not to make aperiodic work unlimited, but to serve it quickly within a bounded budget.

11. How to use sources legally and scientifically

Your source situation is important. Buttazzo is a strong textbook reference, but if you cannot access it legally through HSHL or SpringerLink, you should not write as if you used it. Instead, use accessible university lecture materials and the original paper record, then ask the professor or library if the full paper/book is needed.

- Do not use illegal book PDF websites.
- Use HSHL library, IEEE Xplore, SpringerLink, Google Scholar, official university lecture pages, or professor-provided material.
- For every source, write a short note explaining how it supports your topic.
- Do not copy sentences. Read, close the source, then explain in your own words.
- Cite every technical claim that comes from a source.

12. Milestone 1 writing guidelines

Milestone 1 should not be the final seminar paper. It should show that you technically understand the topic and know what still needs verification. A good 2-4 page briefing can follow this order:

1. Problem statement and motivation: why aperiodic task handling is needed.
2. Core idea: server-based reserved budget for aperiodic requests.
3. Formal model: periodic tasks $\tau_i = (C_i, T_i, D_i)$, server $DS = (C_s, T_s)$, utilization $U_s = C_s / T_s$.

4. Algorithm: replenishment, budget preservation, budget consumption, budget exhaustion.
5. Comparison: Polling Server versus Deferrable Server versus Sporadic Server.
6. Assumptions and limitations: interference, schedulability, parameter choice.
7. Runtime/resource aspects: timers, queues, context switching, bookkeeping.
8. Open questions: what to ask the professor before final paper writing.

Writing rule: For every paragraph in the final Milestone 1 document, ask: Did I understand this and write it myself? If the answer is no, rewrite it after reading the source again.

13. Open questions and checklist

Possible open questions to include in the notebook or ask the professor:

- Should the main comparison be only with Polling Server, or also with Sporadic Server?
- How deep should the schedulability analysis be for a 5-page seminar paper?
- Is a small Python simulation acceptable, or is UPPAAL/C/C++ preferred?
- Should the application example focus on embedded control, communication, or operating systems?
- Can I use open university lecture slides if Buttazzo is not available through HSHL?

Self-check before submitting Milestone 1:

Question	Can I answer it?
Can I explain what aperiodic tasks are?	Yes / partly / no
Can I explain why Polling Server may waste capacity?	Yes / partly / no
Can I define C_s and T_s ?	Yes / partly / no
Can I describe the Deferrable Server algorithm step by step?	Yes / partly / no
Can I explain why budget preservation improves response time?	Yes / partly / no
Can I explain why preserved budget can cause interference?	Yes / partly / no
Can I name at least two reliable sources for verification?	Yes / partly / no

References and source-use notes

[1] J. K. Strosnider, J. P. Lehoczky, and L. Sha, "The Deferrable Server Algorithm for Enhanced Aperiodic Responsiveness in Hard Real-Time Environments," IEEE Transactions on Computers, vol. 44, no. 1, pp. 73-91, 1995. DOI: 10.1109/12.368008. Use: original scientific source for the Deferrable Server algorithm and its motivation.

[2] J. Reineke, "Resource Reservation Servers," Saarland University lecture slides, 2013. Use: clear explanation of Polling Server, Deferrable Server, and Sporadic Server budget behavior.

[3] C. Lu, "Scheduling Aperiodic Tasks," Washington University real-time systems slides. Use: simple explanation that Deferrable Server preserves unused capacity until the end of the current period and can improve aperiodic response.

[4] University of British Columbia / CPEN 432, "Servers for Aperiodic Tasks" slides. Use: additional explanation of aperiodic task servers and limitations.

[5] Masaryk University, "Real-time systems" lecture material. Use: time-demand analysis and schedulability perspective for Deferrable Server.

[6] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, Springer. Use: recommended textbook background if legally accessible through a library or SpringerLink; do not claim use if it is not accessible.

[7] HSHL Real-Time Systems seminar organization slides. Use: seminar requirements: IEEE-style paper, presentation, scientific writing, reference notes, plagiarism rules, and GitHub/version-control workflow.

Final reminder: This document is a study guide. It can help you understand the topic, but your final seminar paper and Milestone 1 text should be written in your own words after verifying the sources.